

Programovací jazyky a překladače

Přednáška 11, poznámky

Jan Voráček

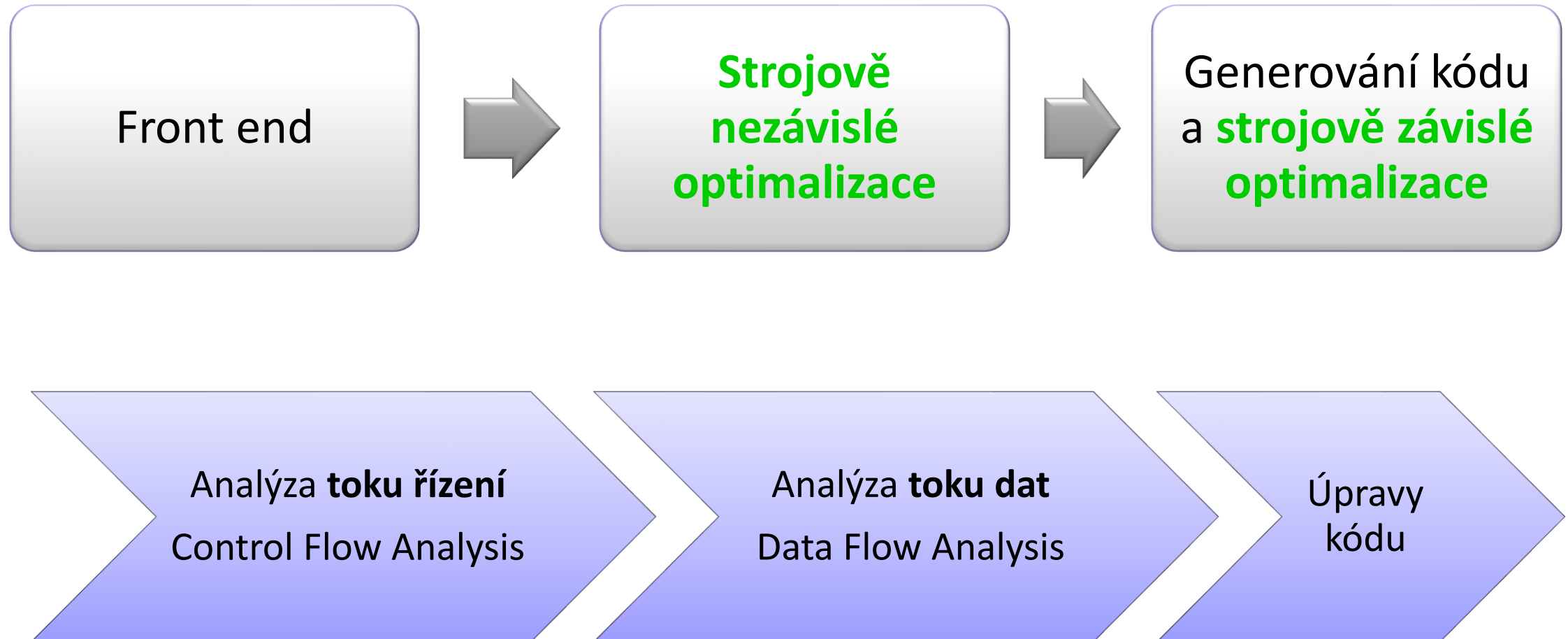
VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Prezentace
 - Strojově nezávislé optimalizace
 - Strojově závislé optimalizace
 - Generování cílového kódu
- Dokončení témat z minulé přednášky
 - Tabulka symbolů
 - Struktura programu za běhu
- **Strojově nezávislé a závislé optimalizace**

- Tradiční cvičení ani přednáška se 22.5. nekoná, studenti se v té době mohou připravovat ke zkoušce, konat její písemnou část nebo samostatně pracovat na přípravě finální sady k zápočtu, případně SP
- Osobní konzultace libovolných témat dne 22.5. jsou možné po předchozí mailové domluvě, realizované nejpozději do 20.5.
- V P3 proběhne 22.5. od 9:30 ústní část předtermínu – viz samostatné instrukce v Moodle

Optimalizace

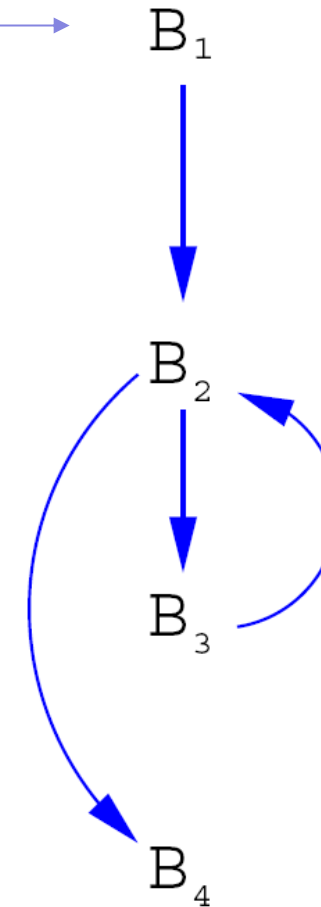


Základní bloky (BB) a řídicí diagramy (CFD)

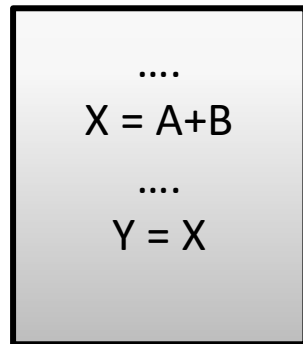
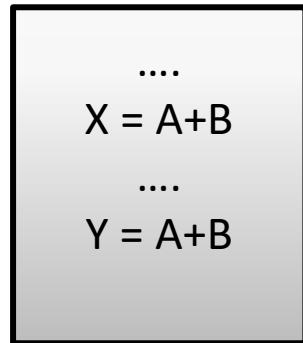
BB: zřetězené souvislé úseky kódu s jednoznačným I/O

CFD: způsob propojení BB

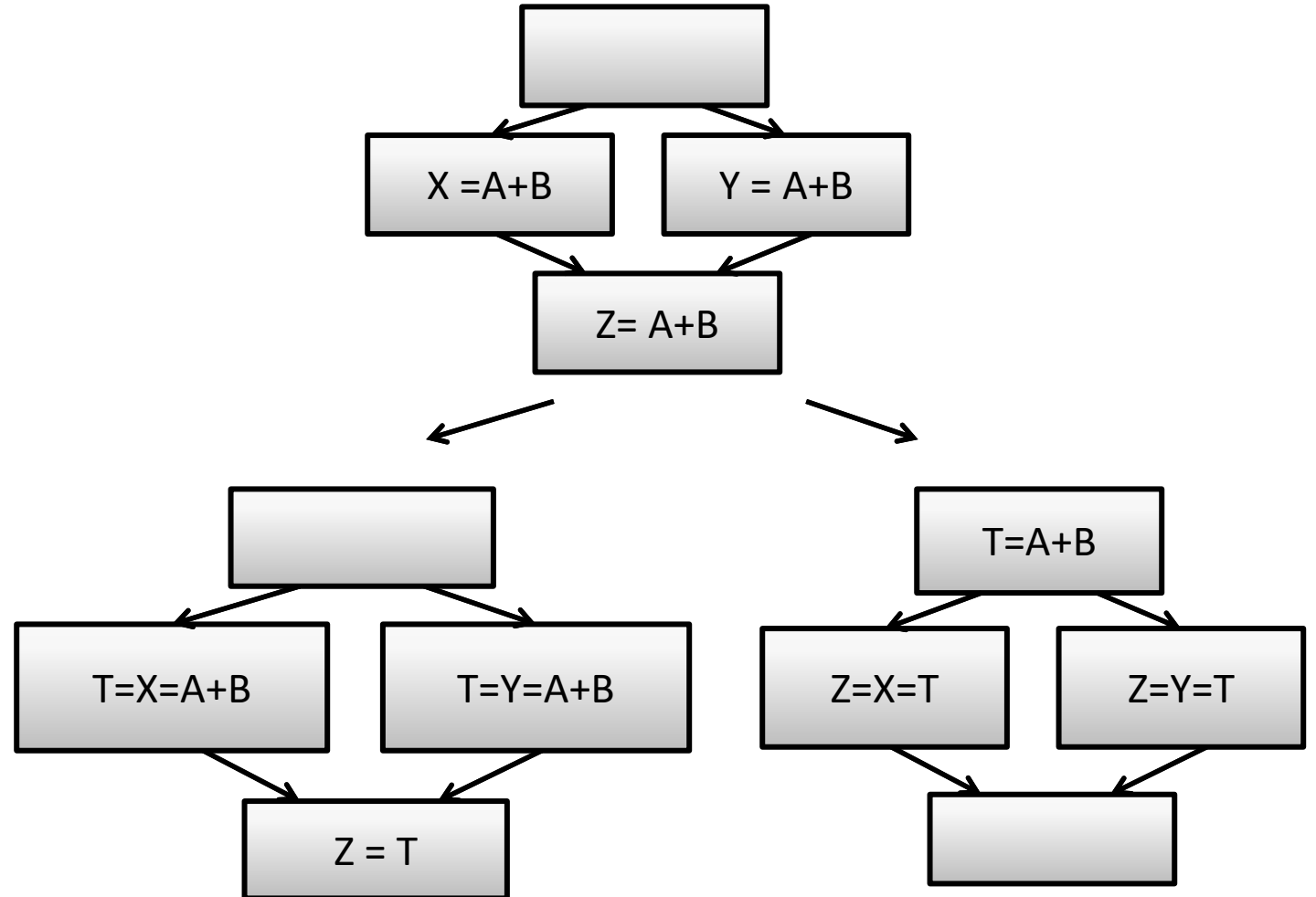
B1	$a = 0$ $b = c + b$
B2	M: $\text{if } (c < a) \text{ goto L}$
B3	$c = c + b$ $a = a * 2$ goto M
B4	L: print c



Lokální (DAG) a globální optimalizace (CFD)

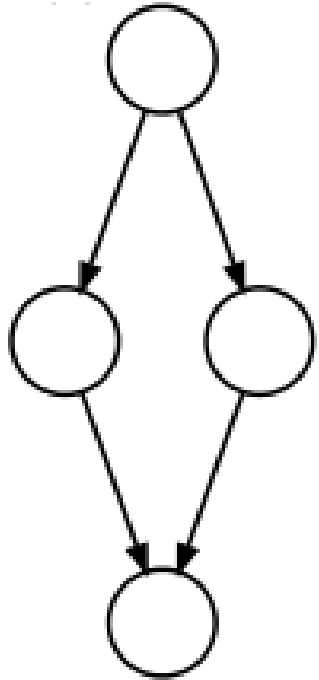


Lokální: uvnitř bloků

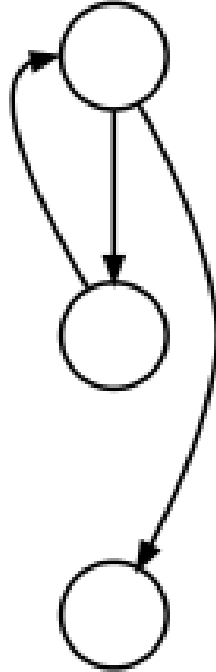


Globální: mezi bloky

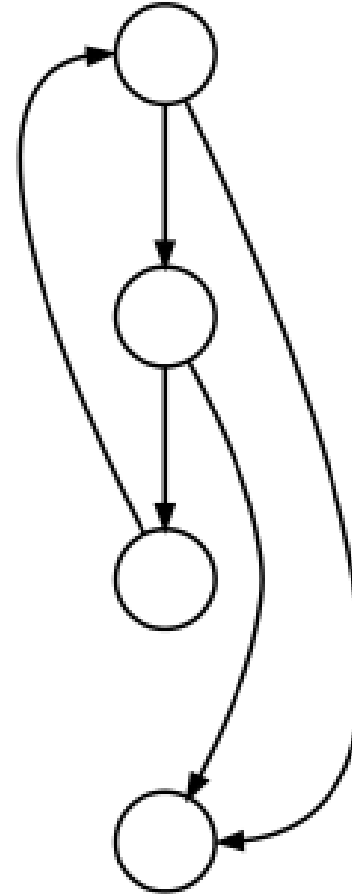
Příklady řídicích diagramů (CFD)



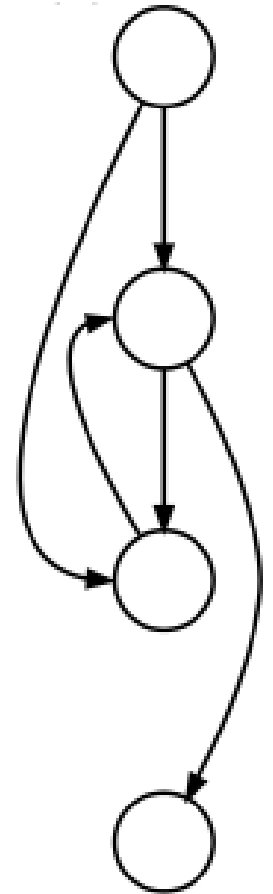
If-then-else



while



while + if



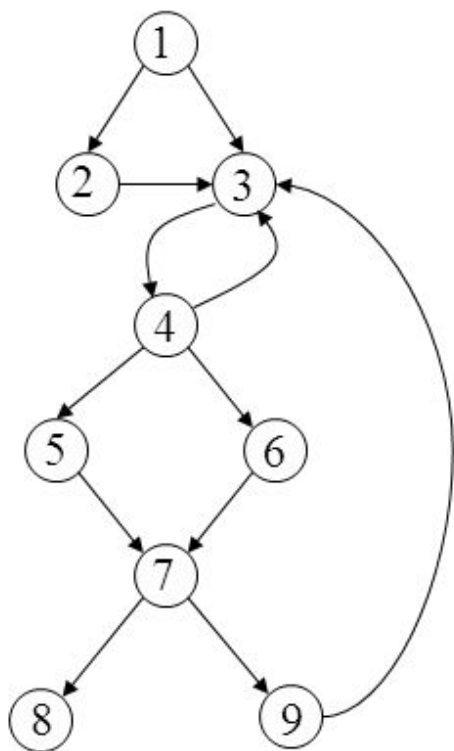
goto + while/if

Zdroj: https://en.wikipedia.org/wiki/Control_flow_graph

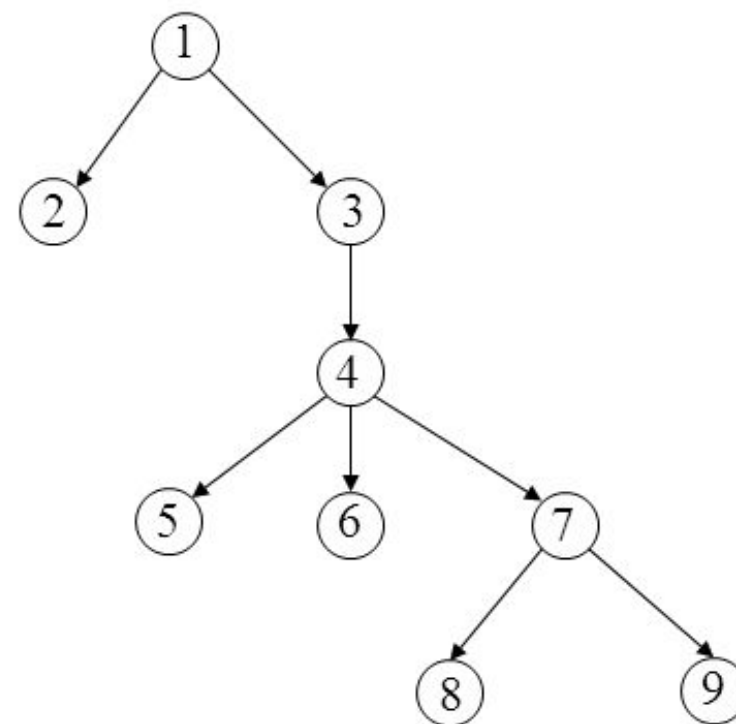
Smyčky v řídicích diagramech

Tok/graf řízení (CFG) a graf dominance (PDG)

Uzel d dominuje uzlu n tehdy, když každá cesta z *počátečního uzlu* do uzlu n musí procházet uzlem d . Uzly jsou v tomto případě základní bloky.



Od grafu řízení ke grafu dominance



Zdroj: [https://en.wikipedia.org/wiki/Dominator_\(graph_theory\)](https://en.wikipedia.org/wiki/Dominator_(graph_theory))

Orientovaný acyklický graf, princip

Pro (orientovaný) acyklický graf (Directed Acyclic Graph, DAG) základního bloku platí v konstrukci/optimalizaci překladačů následující pravidla:

- Listové uzly DAG obsahují data (hodnoty, identifikátory nebo odkazy na ně)
- Vnitřními uzly DAG jsou operátory s odkazy na příslušné operandy
- Nové listy se tvoří pouze pro unikátní hodnoty
- Přiřazovací operátor typu $x:=y$ obvykle vypouštíme z důvodu návaznosti základního bloku do grafu řízení nebo dominance.
- Při konstrukci se vychází z odpovídajícího tříadresového kódu
- Orientace grafu bývá obvykle vstup/výstupní a je důležitá zejména z důvodu demonstrování absence cyklů.

Orientovaný acyklický graf, příklad

DAG pro základní blok reprezentovaný výrazem:

$$(a + b) + (e + (c - d))$$

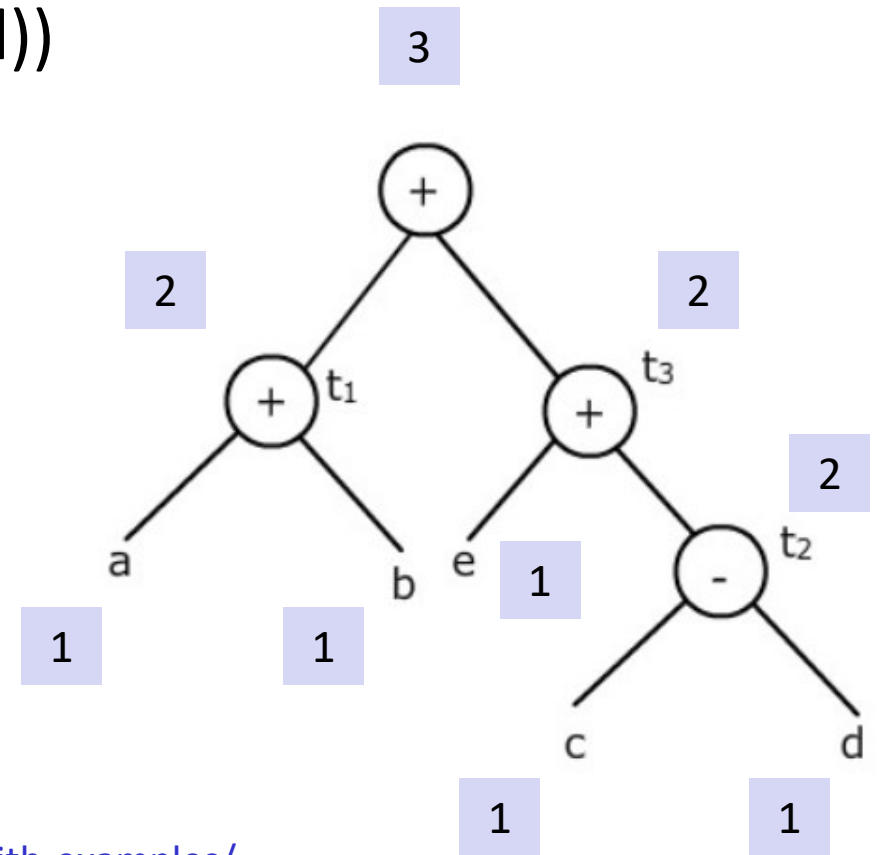
vypadá následovně:

$$t1 = a + b$$

$$t2 = c - d$$

$$t3 = t2 + e$$

$$\text{DAG} = t3 + t1$$



Více příkladů návrhu DAG najdete např. zde:

<https://www.geeksforgeeks.org/directed-acyclic-graph-in-compiler-design-with-examples/>

<https://www.gatevidyalay.com/directed-acyclic-graphs/>

Optimalizace pomocí DAG, příklad 1

$x = (((a+a) + (a+a)) + ((a+a) + (a+a)))$

t1

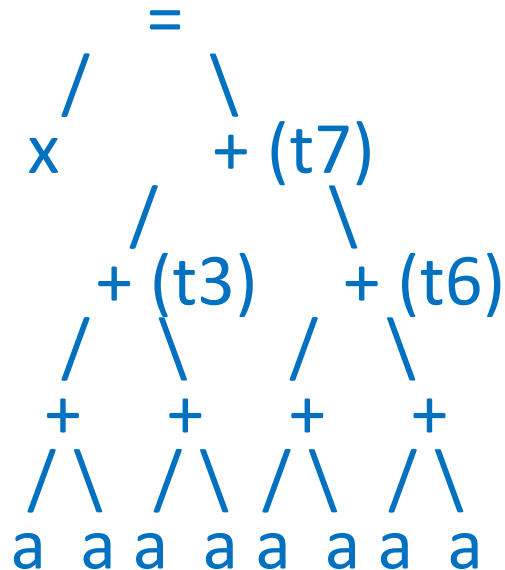
t2

t4

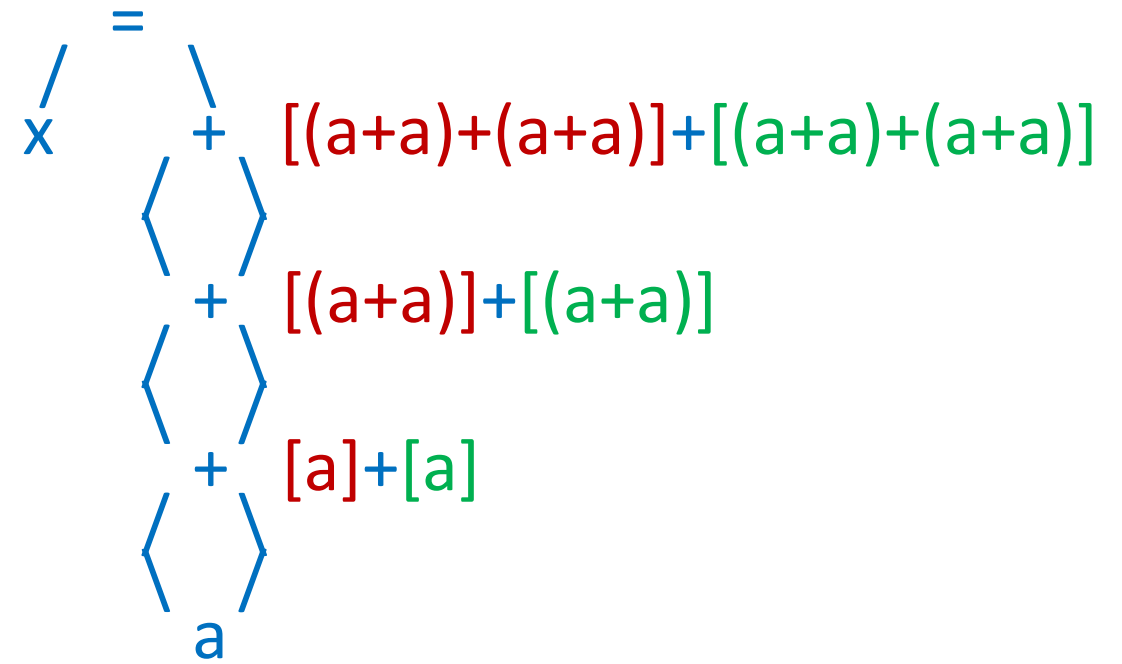
t5

$t3=t1+t2, t6=t4+t5, t7=t3+t6, x = t7$

Syntaktický strom

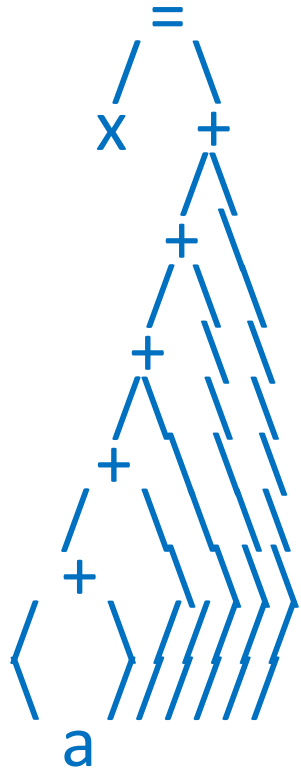


DAG



Optimalizace pomocí DAG, příklady 2 - 4

$x = a + a + a + a + a + a$



$[(a+a+a+a+a)]+[a]$

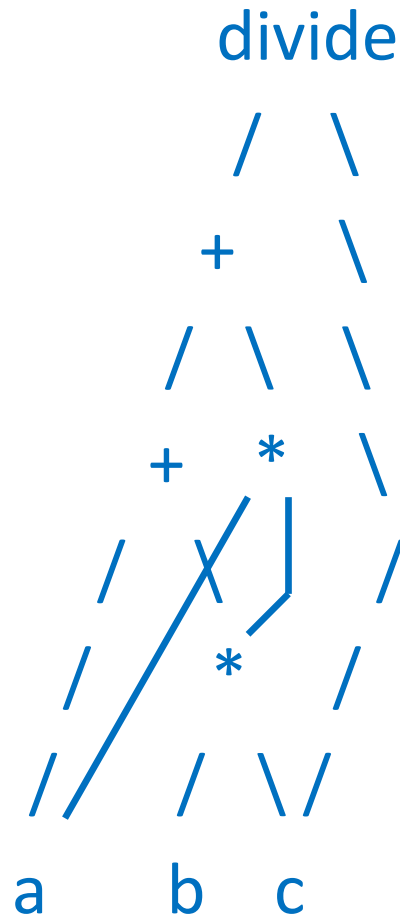
$[(a+a+a+a)]+[a]$

$[(a+a+a)]+[a]$

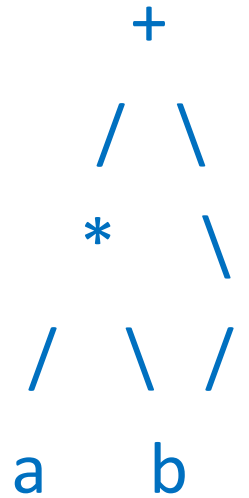
$[(a+a)]+[a]$

$[a]+[a]$

$(a+b*c) + (a*b*c))/c$



$a * b + b$



Optimalizace dočasných proměnných / registrů

Kromě eliminace společných podvýrazů lze DAG použít i ke stanovení optimálního počtu dočasných proměnných nebo registrů. V tomto případě lze využít tzv. „Jeršovovo“¹ číslování (v angličtině též Yershov nebo Ershov numbers), řídící se následující jednoduchou (avšak účinnou) heuristikou:

- Očíslujte každý list jedničkou
- Číslo rodiče s jedním potomkem odpovídá číslu jeho potomka
- Číslo rodiče se dvěma potomky odpovídá:
 - Většímu z čísel obou potomků, jsou-li rozdílná
 - O jedničku zvýšenému číslu jeho potomků, jsou-li stejná

Příklad takového číslování najdete na předchozím obrázku

¹ https://en.wikipedia.org/wiki/Andrey_Yershov

Analýza toku dat, datové diagramy (DFD, Data Flow Diagram/Graph)

- Nalezení **živých a mrtvých proměnných** na vstupech a výstupech jednotlivých základních bloků, respektive mezi nimi
 - Proměnná je živá, bude-li její hodnota použita i v budoucnosti
 - Z toho důvodu je třeba při takové analýze postupovat odzadu (viz následující strana)
- Motivace k této analýze:
 - Program může mít neomezený počet proměnných
 - Procesor má omezený počet registrů
 - Z hlediska rychlosti výpočtu je výhodné mít co nejvíce proměnných v registrech
 - Proměnné, které nejsou živé ve stejnou dobu, mohou využívat tentýž registr
 - Proměnná se stane živou svou deklarací (LHS) a zanikne svým použitím (RHS)
 - Při analýze zdola je formalismus opačný (deklarované proměnné v bloku zanikají, zatímco ty zpracováváné se posouvají nahoru – opět viz následující strana)

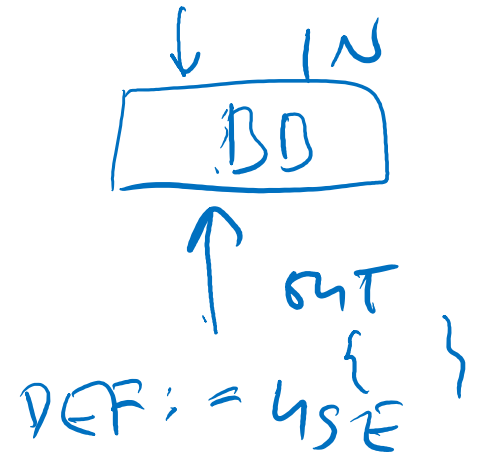
Algoritmus výpočtu živých proměnných

- Jedná se o iterativní algoritmy, které musí zohlednit všechny možnosti průchodu grafem řízení, nebo jednorázové řešení, založené na grafu dominance
- Přímý průchod: pro struktury bez větvení jde pouze o přímé vyhledání nalezení rozsahu platnosti proměnných (viz téma optimalizace TAC/alokace registrů)
- Zpětný průchod: určení množin IN a OUT s přihlédnutím k řídicí struktuře. Začínáme od stavu OUT, který je na konci vždy známý:

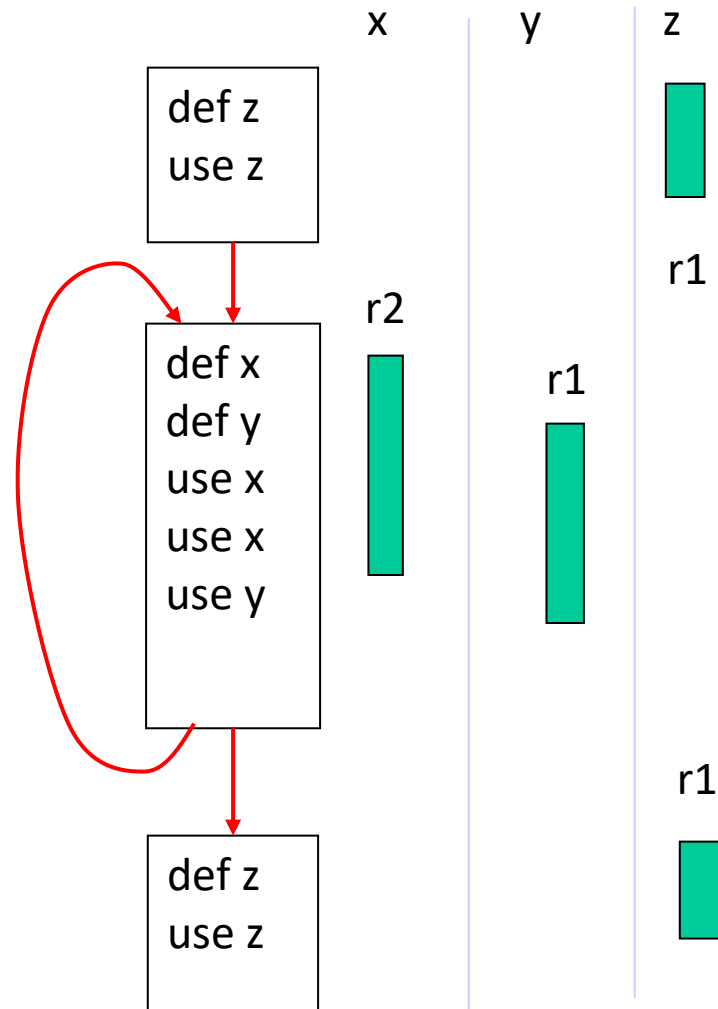
$$IN = (OUT - DEF) + USE,$$

kde:

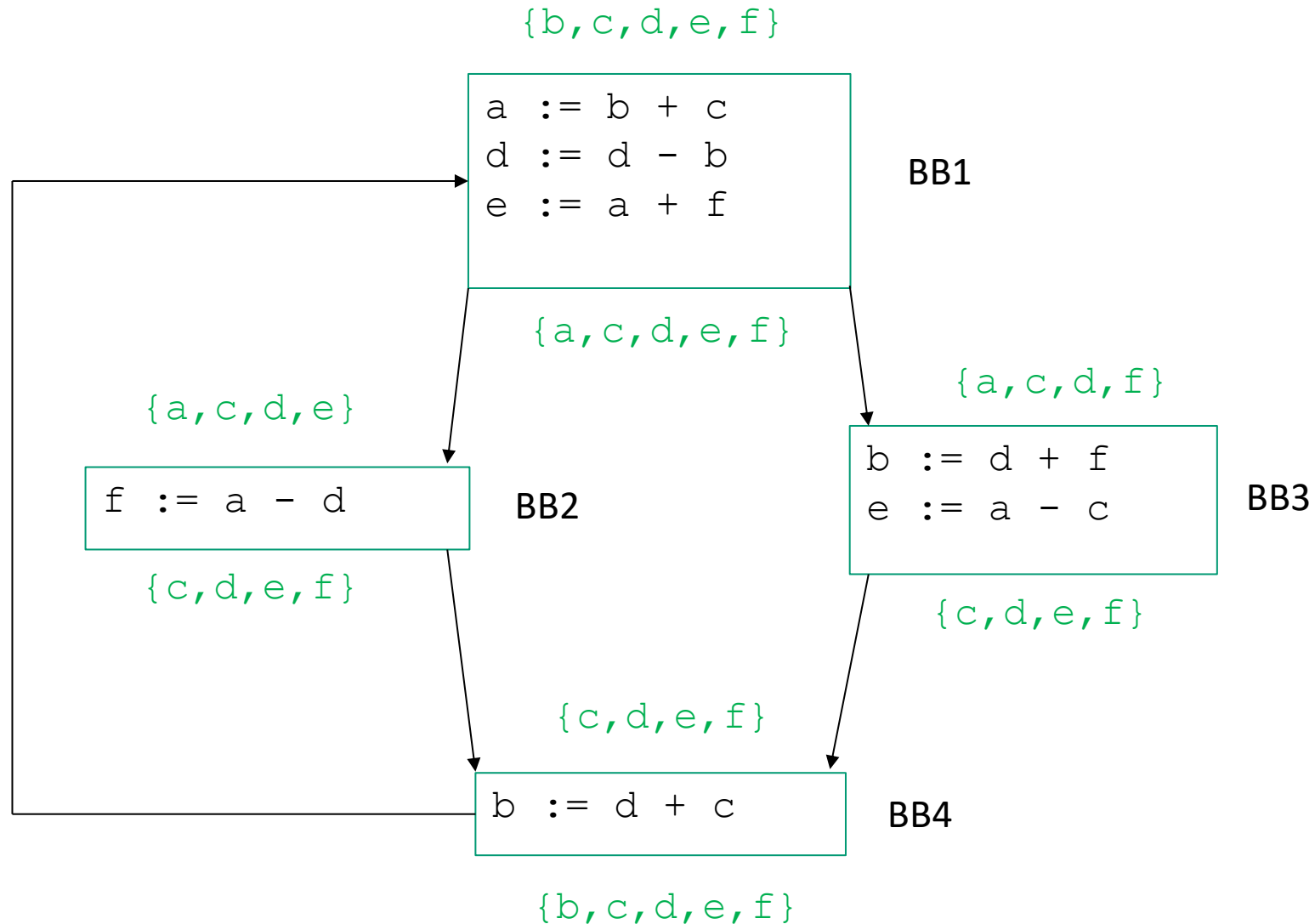
- DEF: je uvnitř bloku nově definovaná proměnná (L-value, LHS)
- USE: argumenty, vztahující se k proměnným DEF (R-value, RHS)



Alokace registrů



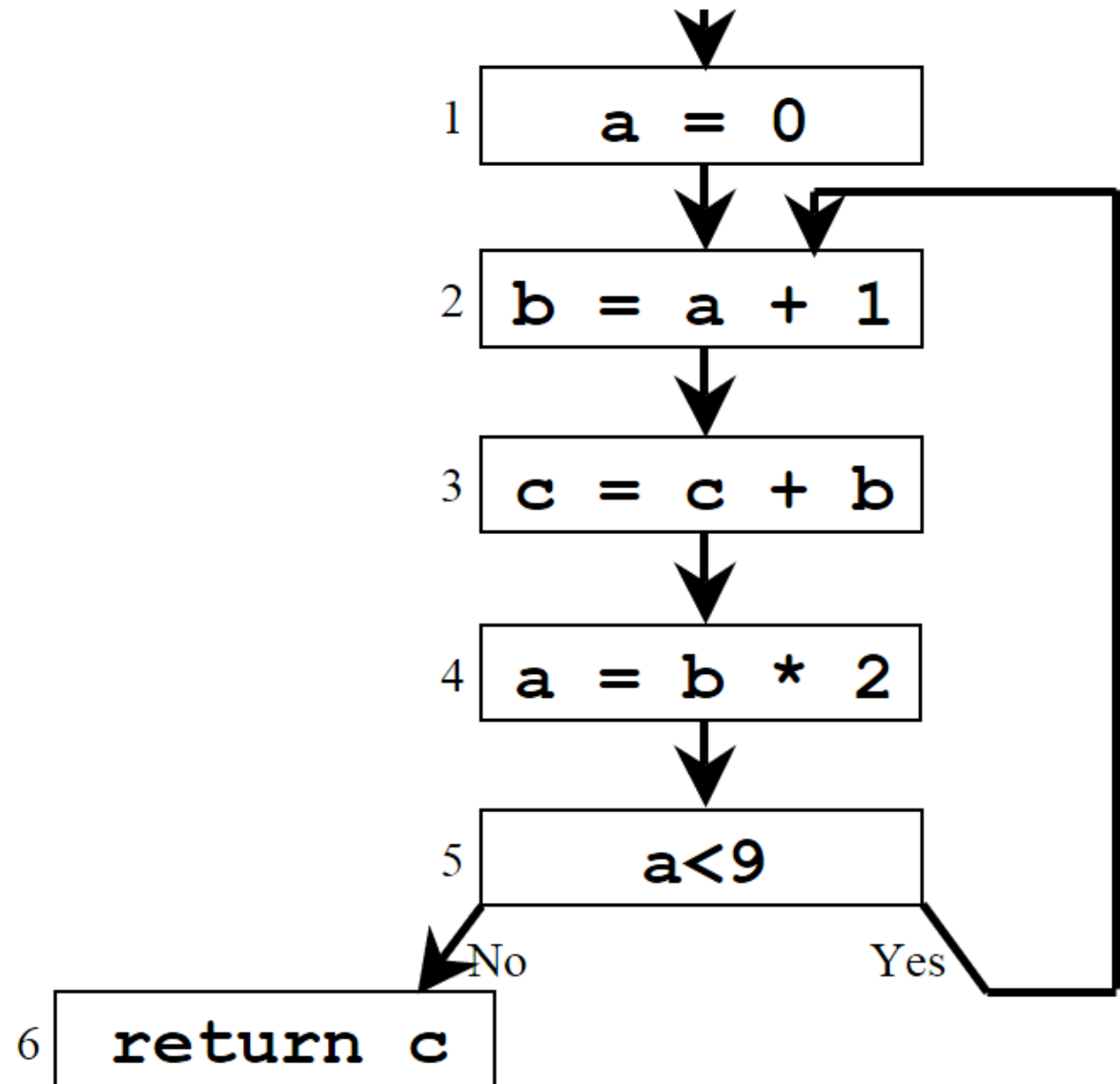
CFG s vyznačením platnosti proměnných



Kolik registrů bude třeba pro překlad?

IN = {c}

```
1      a := 0;  
2  L1: b := a + 1  
3      c := c + b  
4      a := b * 2  
5      if a < 9 goto L1  
6      return c
```



Rozsah platnosti proměnných

- Proměnná je živá, bude-li použita kdykoli v budoucnosti
- Využití: alokace registrů (program má obvykle větší počet proměnných než má procesor registrů)

Jaký je rozsah života proměnné b ?

2 -> 3 -> 4

Jaký je rozsah života proměnné a ?

1 -> 2 a 4 -> 5 -> 2

Proměnná a není živá ve krocích 2 -> 3 -> 4

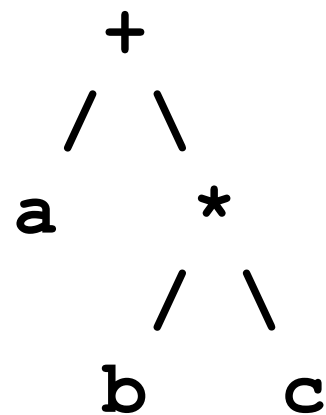
Jaký je rozsah života proměnné c ?

1 -> 2 -> 3 -> 4 -> 5 -> 2 a 5 -> 6

Proměnné a a b mohou sdílet stejný registr.

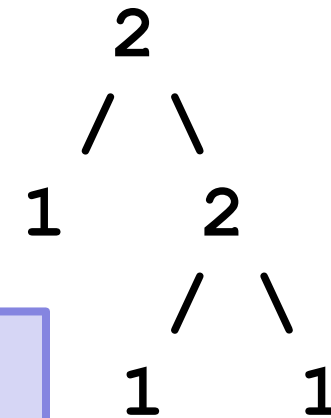
Alokace registrů, ASU schéma

Abstraktní syntaktický strom



---->

Registrový strom
(čísla určují počet registrů)



Odpovídající kód:

```
load b R1
load c R2
mult R1 R2
load a R1
add R1 R2
store R2 x
```

Příklady strojově nezávislých úpravy kódu

- Zjednodušení řídicí struktury programu
 - Smyčky
 - Návěští
 - Logické výrazy
- Přesun kódu
- Eliminace strukturně shodných výrazů
- Algebraické a logické optimalizace (vyčíslení)
- Eliminace nedostupného kódu
- Redukce výpočetní náročnosti

Příklady optimalizace 1

- Přesun kódu

```
while ( a < b ) {  
    c = 10 * d;  
    a = a + c;  
}
```

- Eliminace strukturně shodných výrazů

```
a = b + c  
b = a - d  
c = b + c  
d = a - d
```

Příklady optimalizace 2

- Eliminace „mrtvého“ nebo nedostupného kódu

a = b + c

b = d + e

e = b + f

a = e + x

L1:

goto L1

a = a + b

- Zjednodušení řídicí struktury programu

goto L1

a = b + c

b = a - d

c = b + c

L1: goto L3

L2: e = b + f

a = e + x

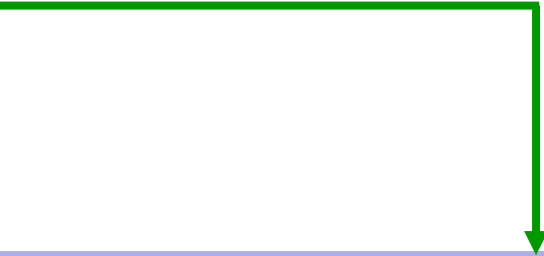
L3:

Příklady optimalizace 3

Náhrada „pomalých“ a operátorů a efektivní indexace:

$16 * x \rightarrow x \ll 4$

```
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        a[n*i + j] = b[j]
```



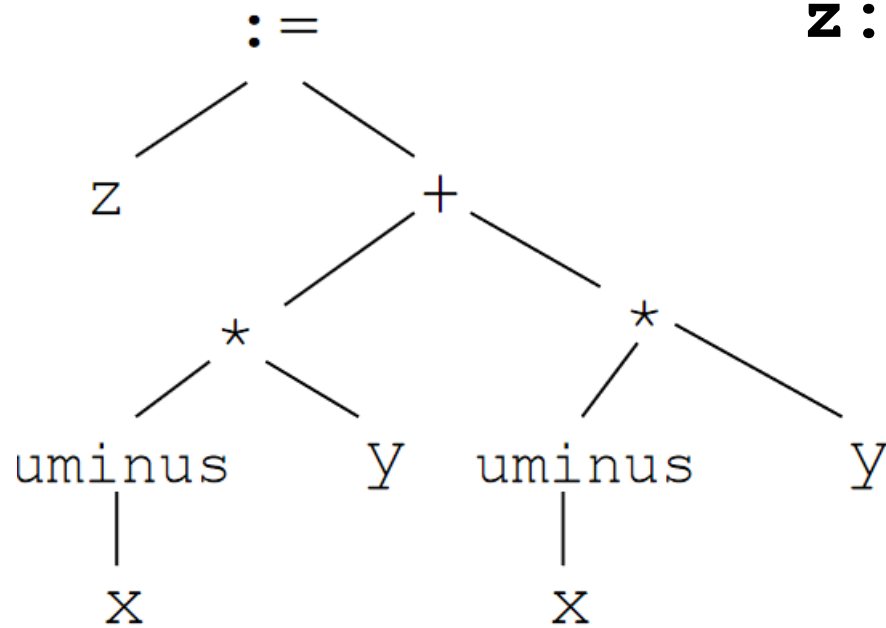
```
int ni = 0;  
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
    ni += n;  
}
```


Příklady optimalizace 4: AST a DAG pro daný výraz

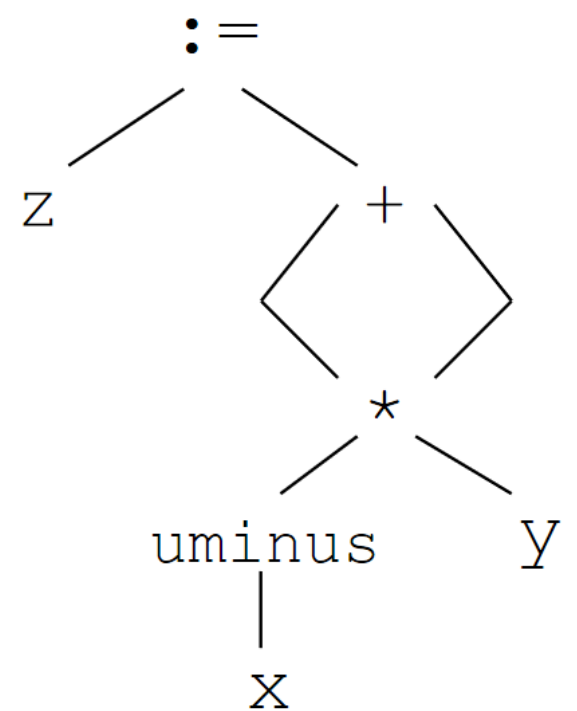
AST: abstraktní syntaktický strom (abstract syntax tree)

DAG: přímý acyklický graf (direct acyclic graph)

$z := -x*y + -x*y$

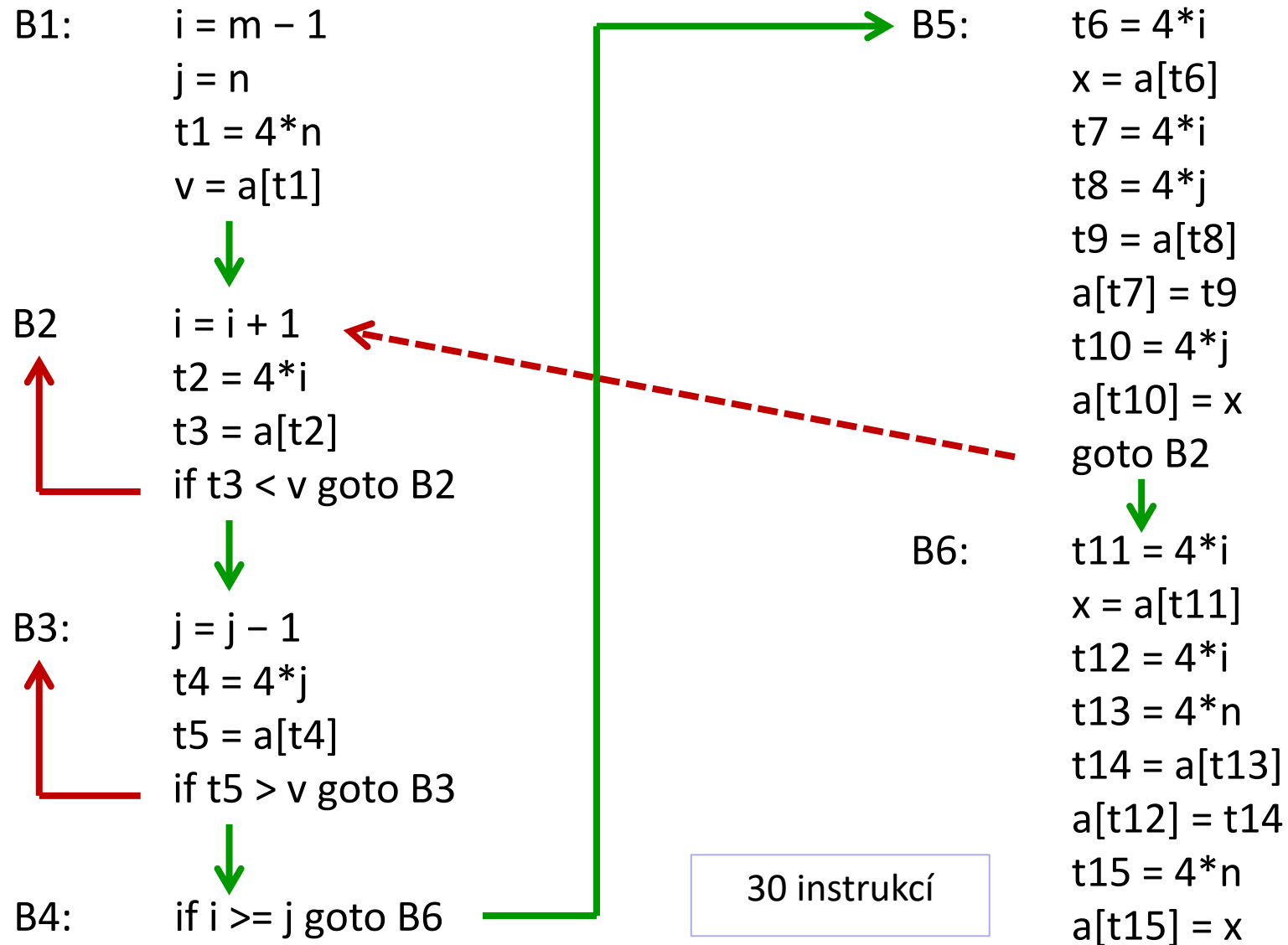


AST

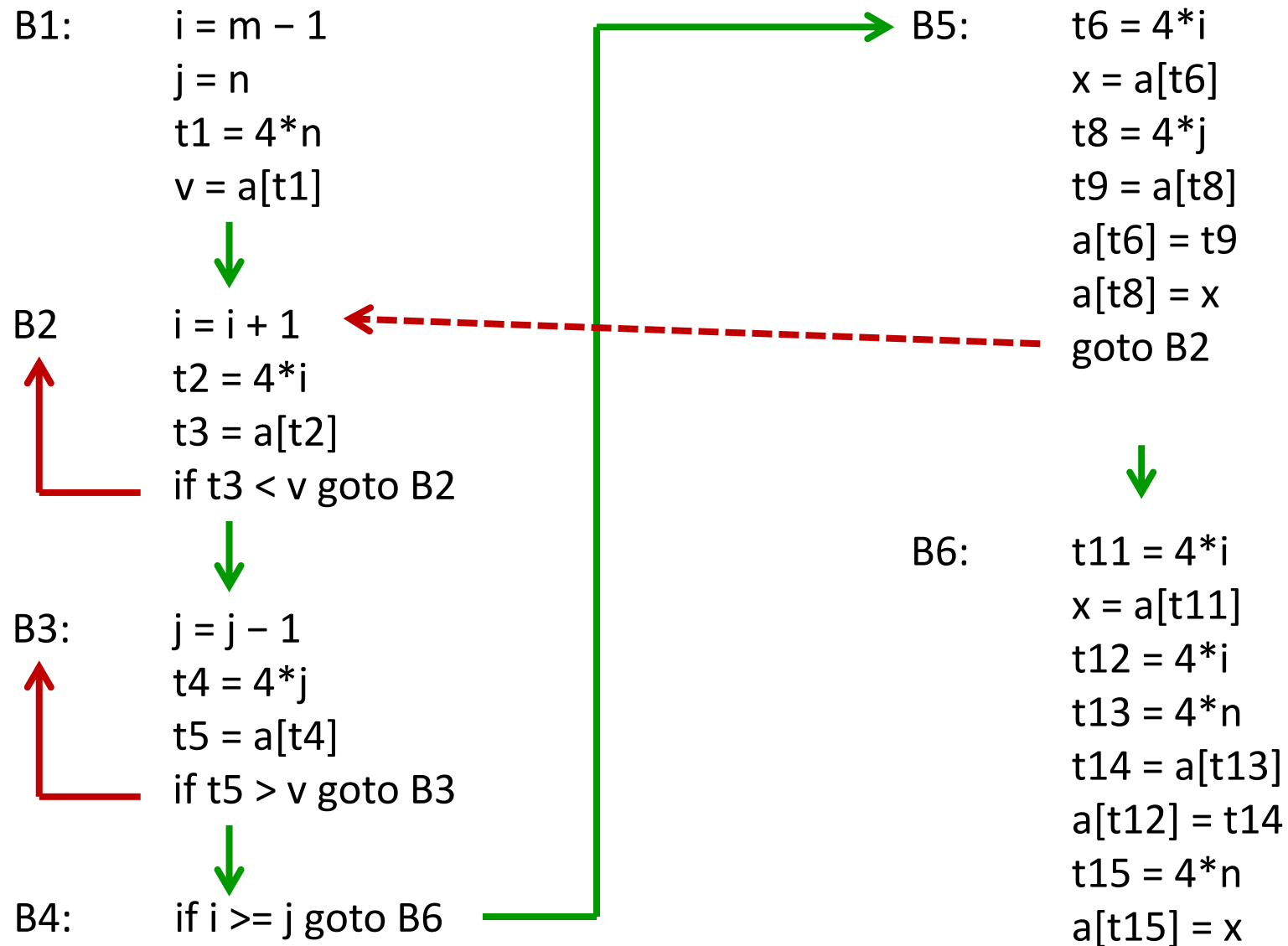


DAG

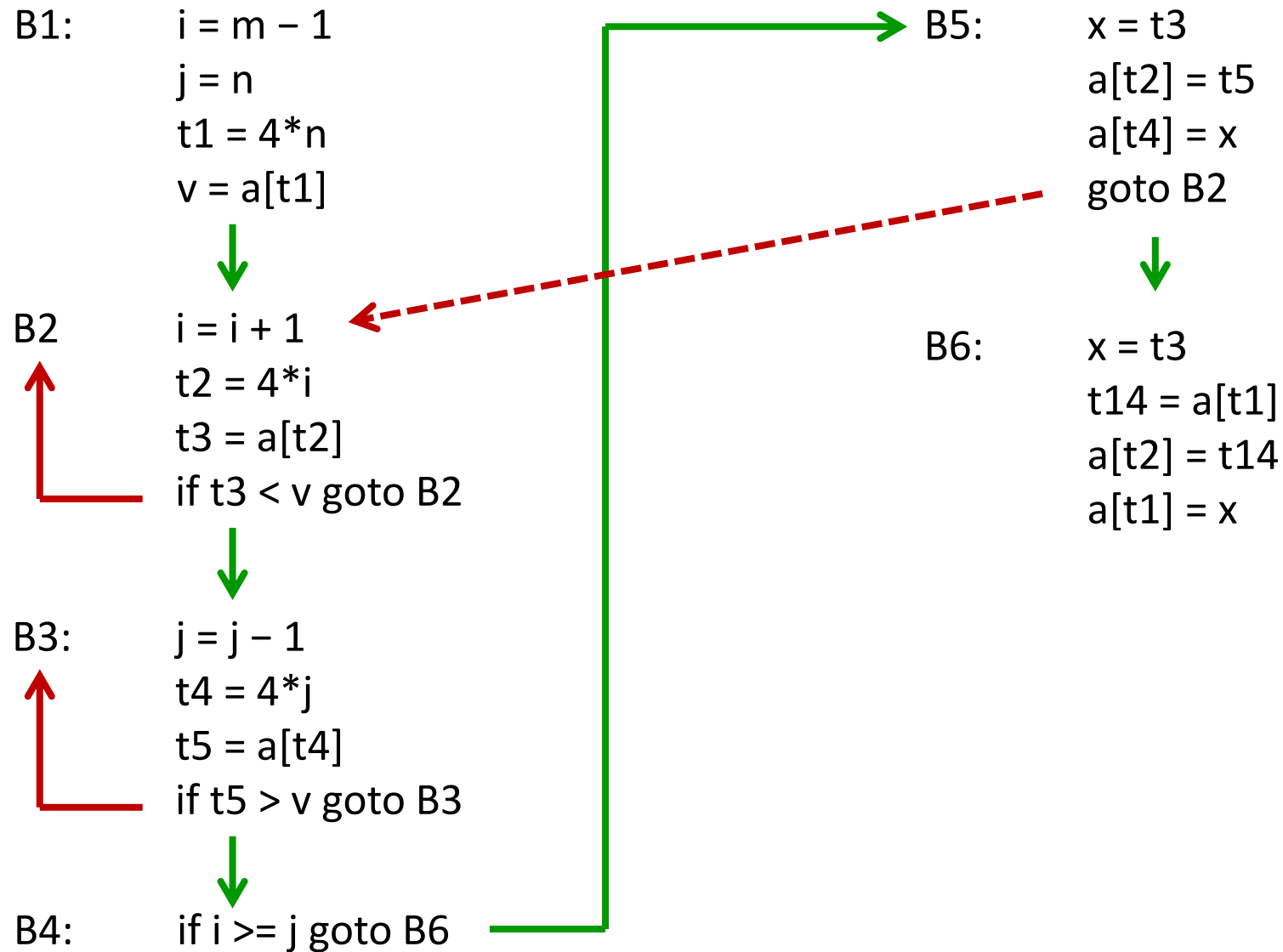
Optimalizace, příklad - základní řídicí diagram



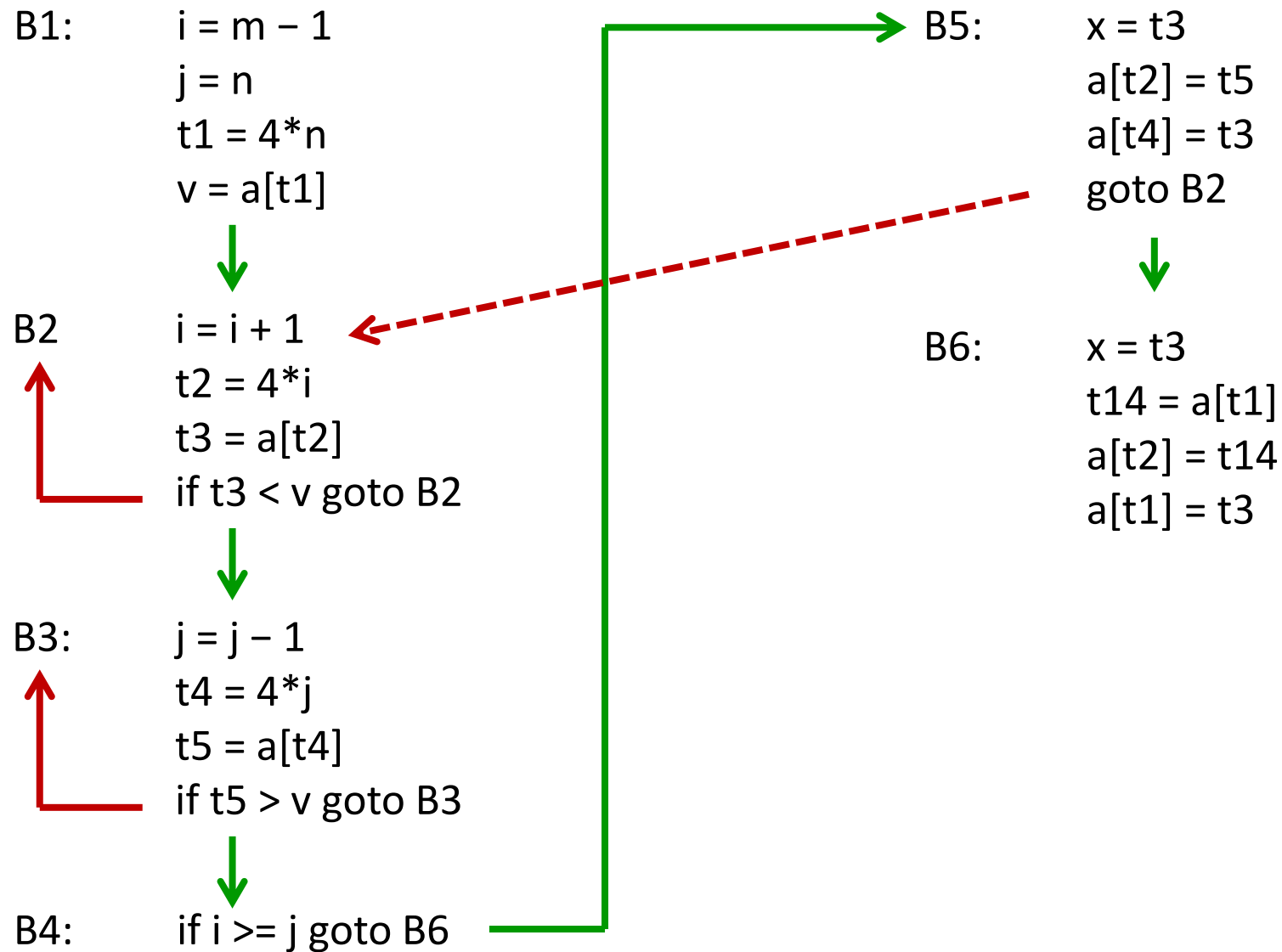
Lokální optimalizace v B5



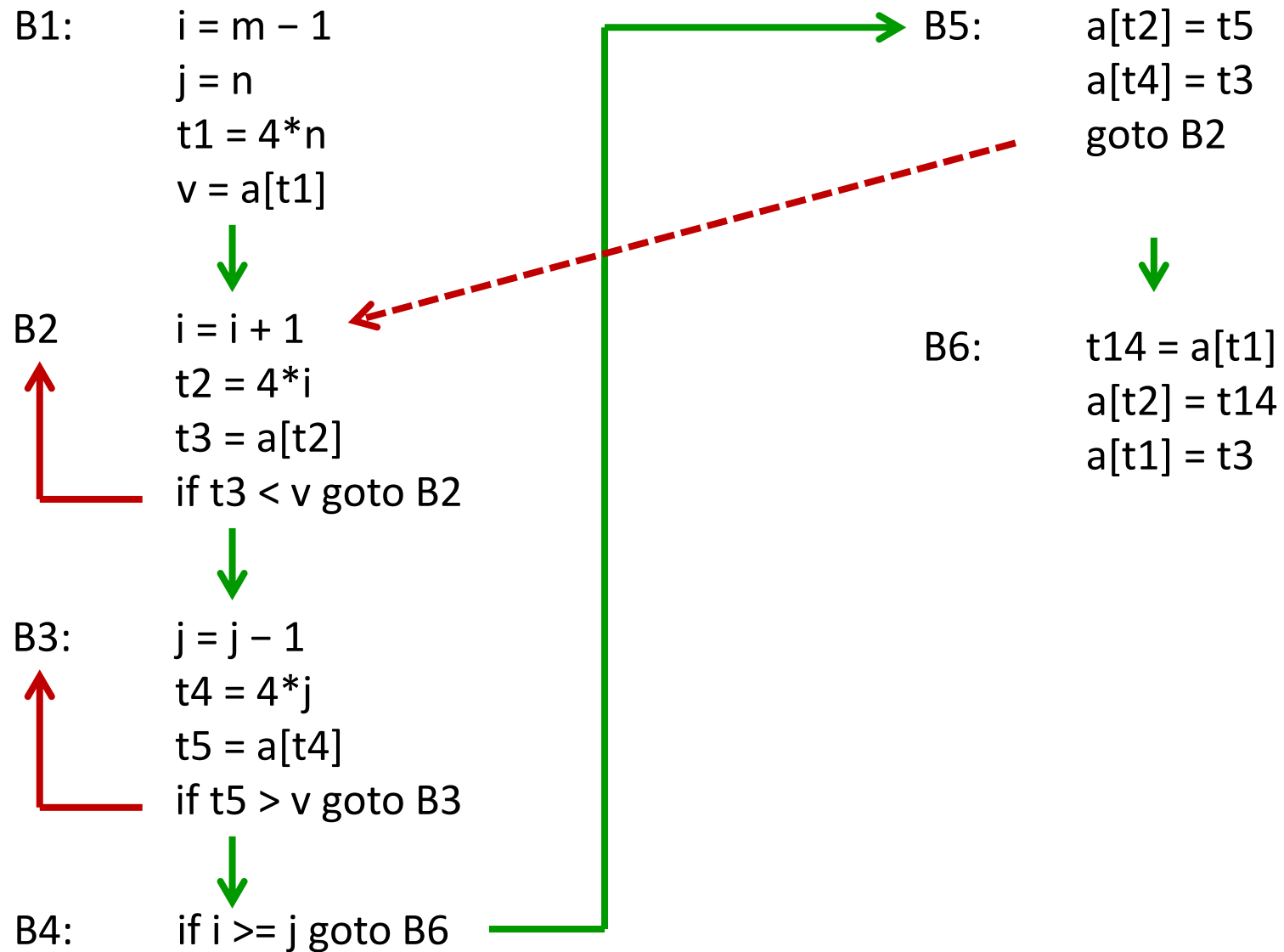
Optimalizace mezi bloky



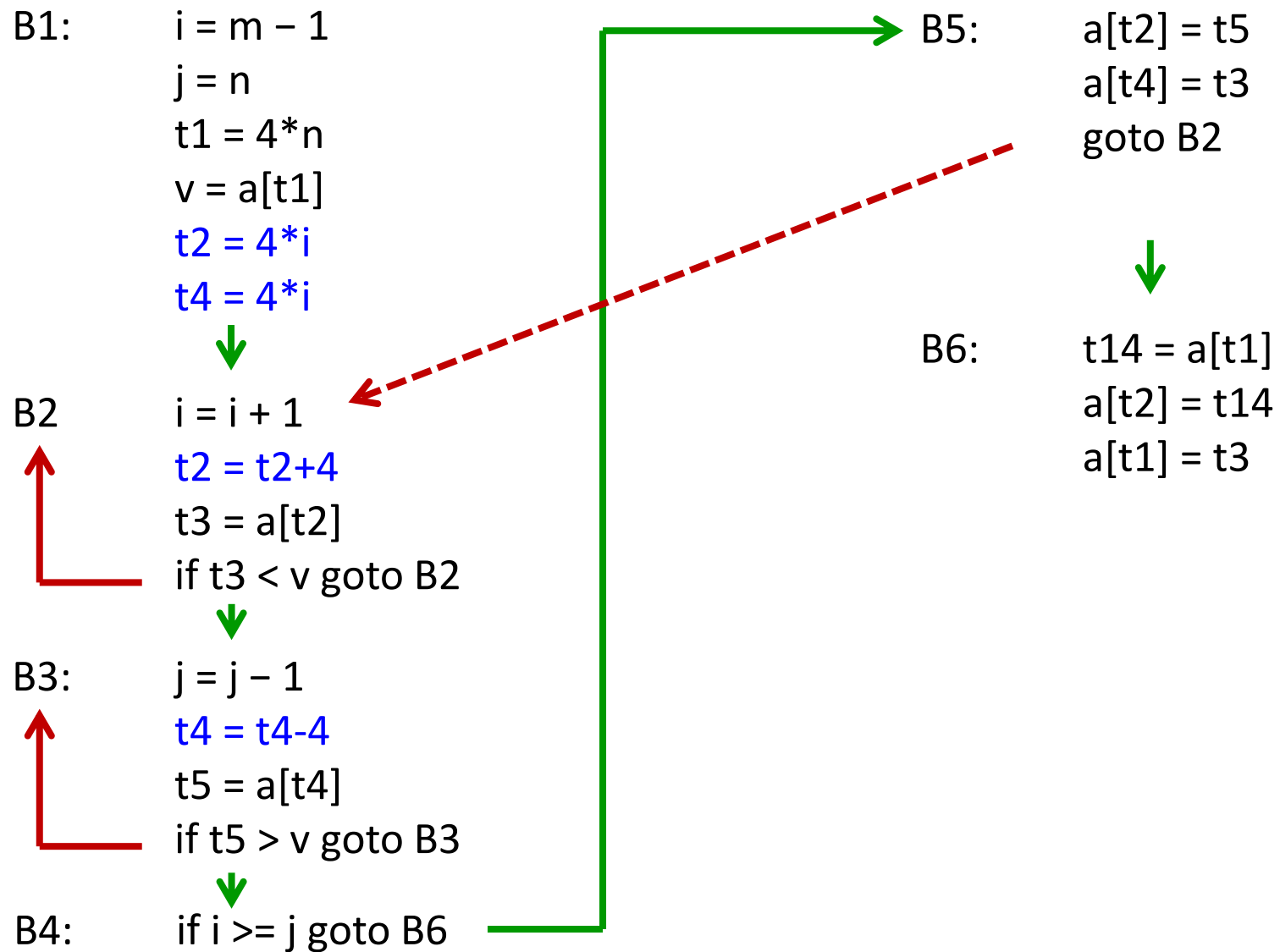
Kopírování hodnot v blocích B5 a B6



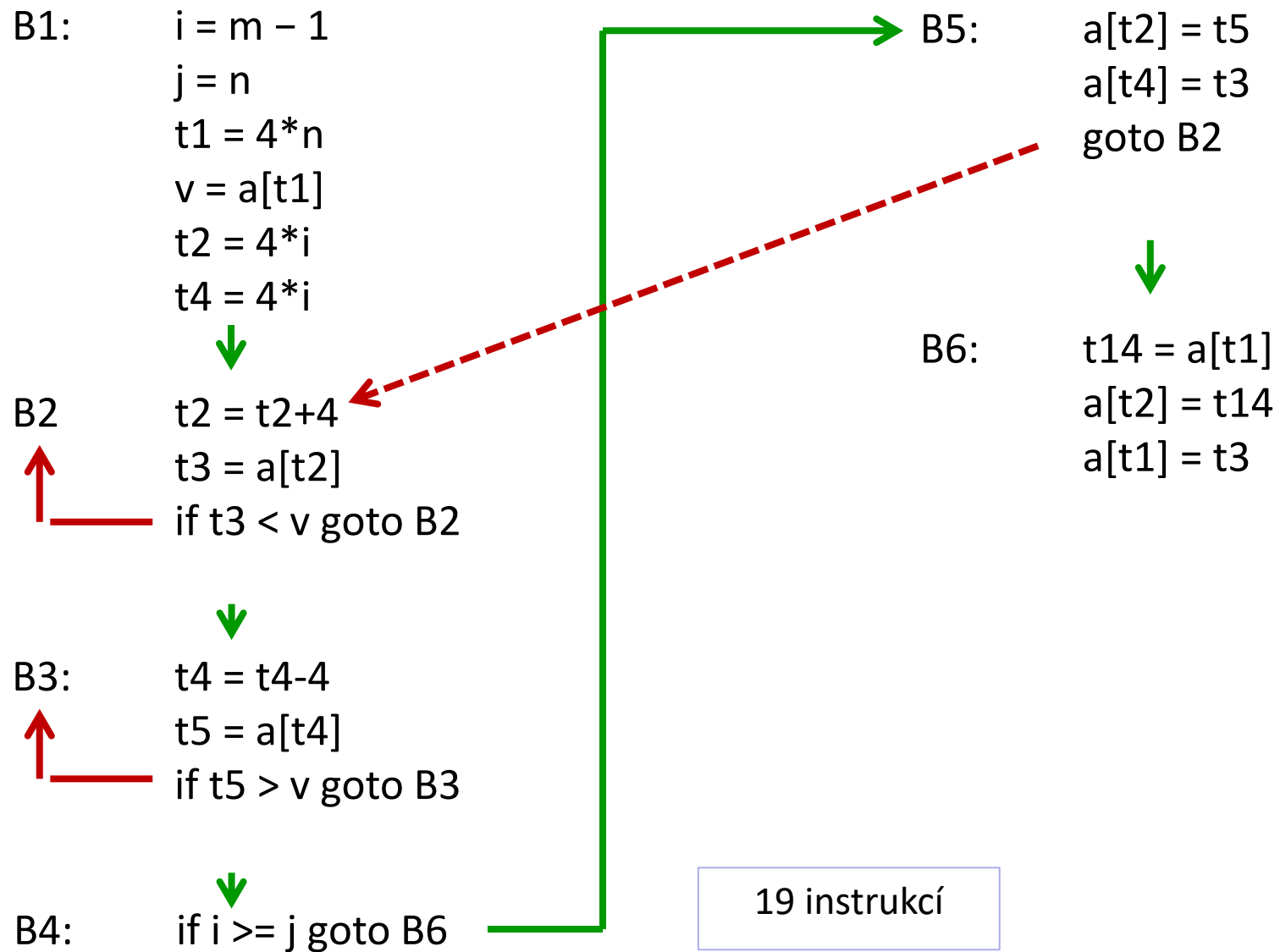
Eliminace nedostupného kódu



Redukce výpočetní náročnosti



Odstranění neaktivních proměnných



Strojově závislé optimalizace

- V této fázi obvykle neprobíhá analýza toku řízení (CFA) ani analýza toku dat (DFA)
- Zohledňují architekturu procesoru
 - CISC (Complex Instruction Set Computer)
 - RISC (Reduced Instruction Set Computer)
- Maximalizace využití (alokace) dostupných registrů
- Přeskupení kódu z důvodu paralelizace
- Na úrovni instrukční sady pracují s malými souvislými úseky kódu (**peephole optimization**)
 - Náhrada jednotlivých instrukcí nebo jejich skupin

Peephole optimization

a := b + c

d := a + e

MOV b, R0

ADD c, R0

MOV R0, a

MOV a, R0

ADD e, R0

MOV R0, d

Obsluha dvou sousledných procedur

PUSH AF		PUSH AF
PUSH BC		PUSH BC
PUSH DE		PUSH DE
PUSH HL		PUSH HL
CALL _ADDR1		CALL _ADDR1
POP HL		
POP DE		
POP BC		
POP AF	---->	
PUSH AF		
PUSH BC		
PUSH DE		
PUSH HL		
CALL _ADDR2		CALL _ADDR2
POP HL		POP HL
POP DE		POP DE
POP BC		POP BC
POP AF		POP AF

Zdroj: https://en.wikipedia.org/wiki/Peephole_optimization

- Funkce: pouze deklarace typů a jejich instalace do TS s prázdnou příkazovou a výrazovou částí
- TS: pole typovaných proměnných (*char** jméno, *int* typ)
 - Funkce *addsymb*, *symprint*
 - Dědění: *variablelist*: *ID {addsymb(\$<type>0, \$1);}*
- Vstupy: jména proměnných a konstantní řetězce včetně typů
 - *int* = 0, *char* = 1, *float* = 2
- Typy: viz TS
- Výstup ze *stdin*: obsah TS s příslušnými typy po ukončení vstupu

- Funkce: generátor tříadresového kódu pro AV a příkazy bez jejich vykonání
- TS: 2D statické pole
 - Obsahuje programové proměnné, hodnoty, dočasné proměnné, návěští
- Inkrementované překladové typy:
 - Čtveřice (quadruples), reprezentované statickým polem struktur
 - Kód operátoru [0], oba operandy [1, 2] a výsledek [3]
 - TAC se tvoří paralelně se čtveřicemi na základě obsahu tabulky symbolů
 - Dočasné proměnné (*gettemp*, generují se při tvorbě AV)
 - Návěští (*getlabel*, generují se při realizaci řídicích příkazů)
- Vstupy: řetězce reprezentující čísla a proměnné
 - Funkce *symlook* je instaluje do TS, funkce *makequad* z nich vytváří mezikód
 - Proto ani nepotřebujeme speciální typy (*yylval = char**)
- Výstup ze *stdin*: čtveřice, TAC a obsah TS po ukončení vstupu